

CHAPTER 8 OF 13 • PART FOUR

The Scale Kit

Every scaling change is one of four moves. Little's law sizes the pool before anyone has to guess.



Thirteen chapters, gathered into six movements.

01 A Tuesday — one engineer, wake to sleep

02 The words — naming what you just watched

03 Protagonist and antagonist — where a design starts

04 Solution types — which answer is wanted

→ **Six kits — data, compute, network, scale, reliability, security**

06 Two designs, then the map back — Instagram, a parking app, and what you keep

WHERE THIS SITS

The cache and the replica return here as moves, not stores.

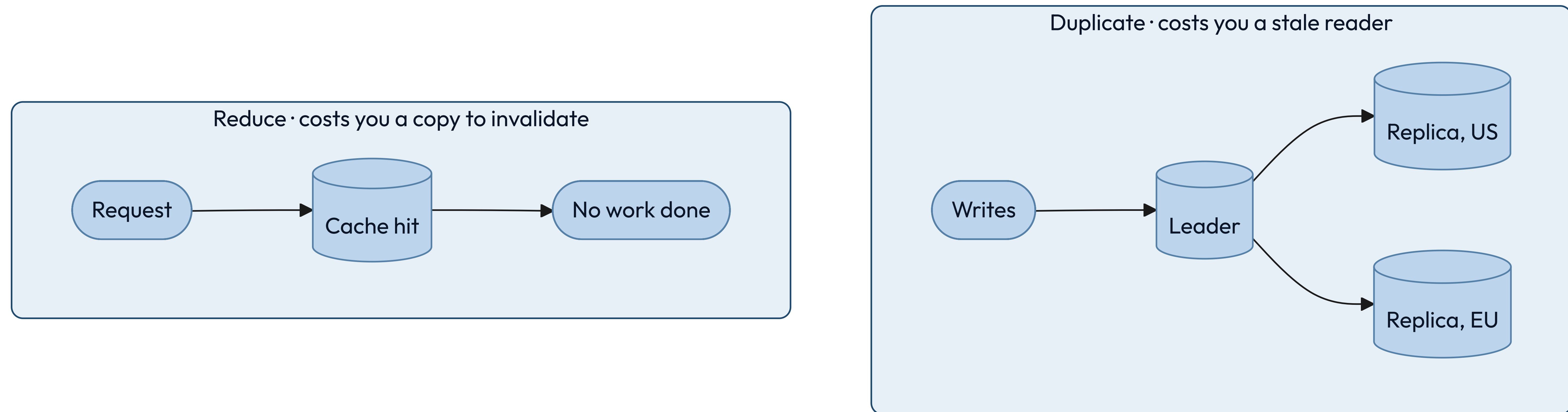
Chapter five introduced the cache and the replica as places data lives. Here they are things you do when load grows. Four moves cover every scaling change: reduce, duplicate, defer, spread. This chapter assumes the data kit's copies and the network kit's latency budget, and it ends on the arithmetic that sizes a pool.

SCALE

Every scaling change is one of four moves: reduce, duplicate, defer, spread.

SCALE KIT • ADDING A COPY

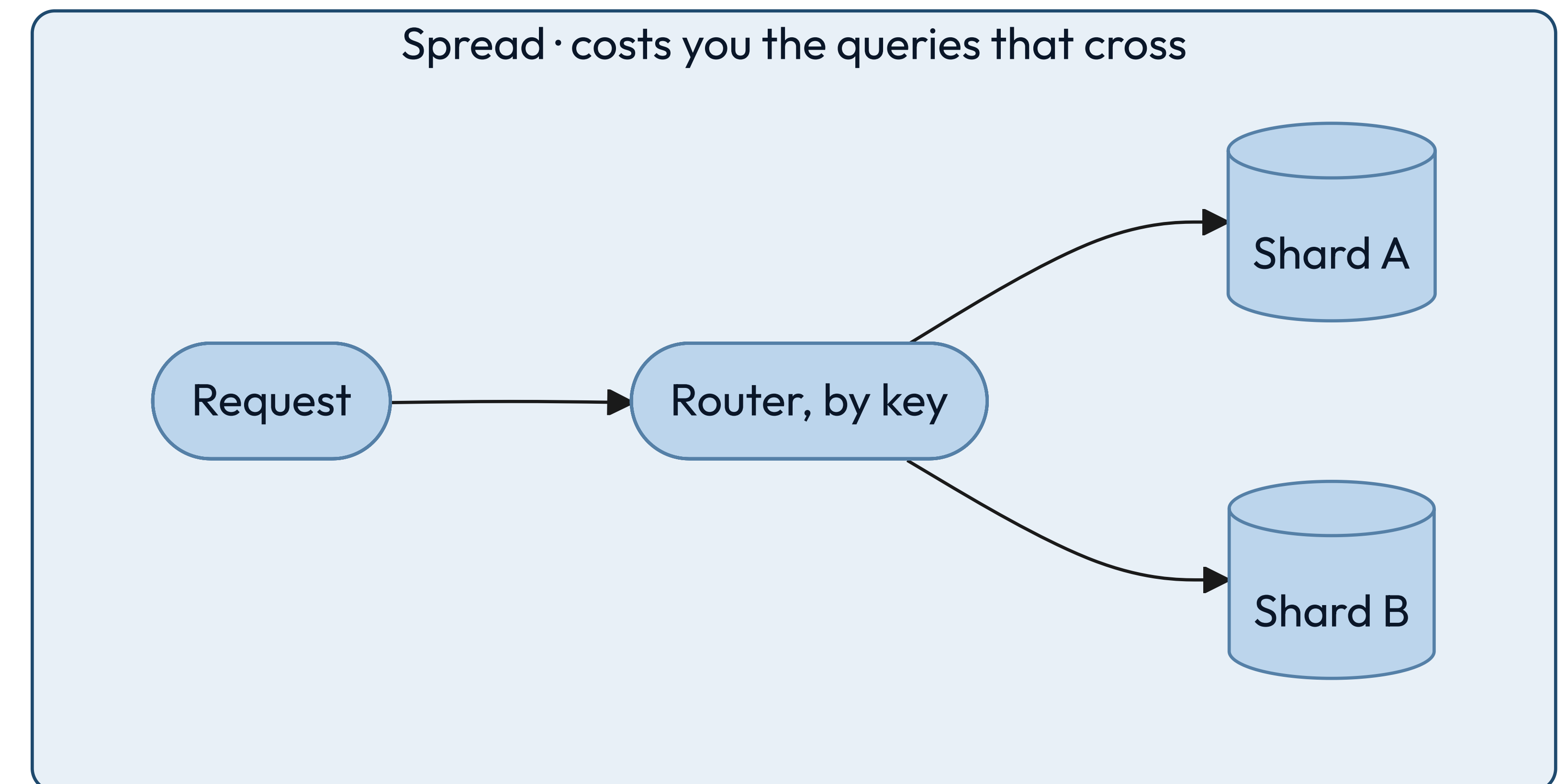
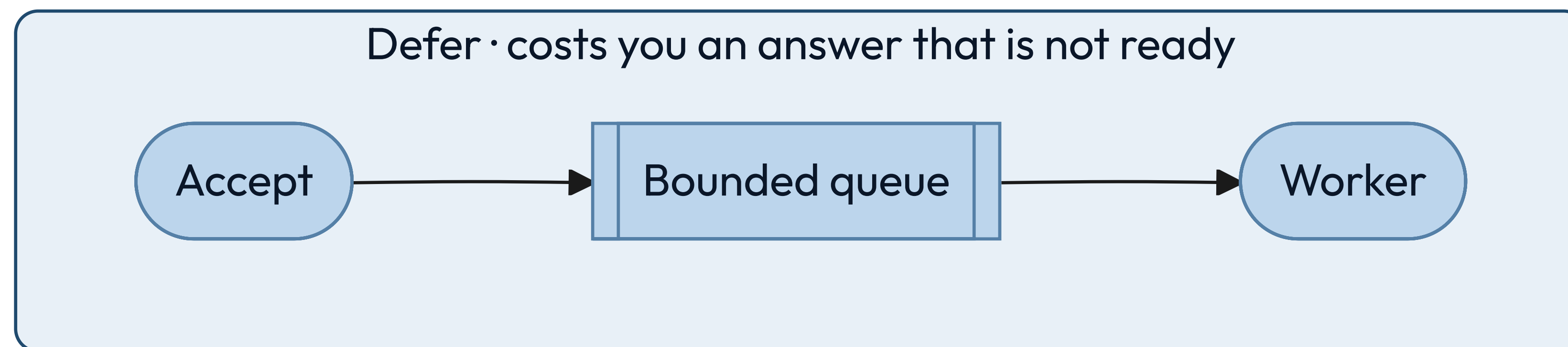
Reduce and duplicate both add a copy of something you already had.

**KEY INSIGHT**

Both bills arrive in the same currency: something you are reading is now out of date.

★ You met both of these as stores — the cache was a data choice, the replica a consistency choice. Here they are again as moves. Distance is why: the only way to spend less of it is to keep a copy nearer.

Defer and spread move the work instead — to later, or somewhere else.



KEY INSIGHT

Both bills arrive as an answer you cannot have right now: not yet, or not from here.

Little's law ties concurrency, throughput and latency together.

$$L = \lambda W$$

WHERE

- L — requests in flight at once
- λ — arrivals per second
- W — seconds each one spends inside

USING IT

Multiply arrivals by seconds inside to get requests in flight.

Your service takes 2,000 requests a second, and each one spends 50 milliseconds inside. Convert first — 50 milliseconds is 0.05 seconds — then multiply: $2,000 \times 0.05 = 100$.

A hundred requests sit inside your service at every instant. If your server hands each request a thread while it runs, that is 100 threads. It is a floor, not a preference: while those two numbers hold, no setting makes it 80.

An idle machine can still be the thing everyone waits for.

Maya's build fleet ran two builds a day at twelve minutes each, so it sat idle almost all day. That tells you it has spare capacity. It tells you nothing else.

Her reviewer was the bottleneck: one person, five pull requests taken in order, awake for one of the hours that mattered. The critical path is the chain of steps that decides when the work finishes. Idle time never tells you whether something sits on it.

WHEN IT GOES WRONG

Tripled latency breaks a bounded pool and an unbounded one in opposite directions.

Latency triples to 150 milliseconds. Leave the pool unbounded and the arithmetic runs forward: $2,000 \times 0.15$ is 300 in flight where you sized for 100. Each one holds a thread, a buffer and a connection, so the machine runs out of memory.

Bound it at 100 and concurrency cannot rise, so throughput falls instead: $100 \div 0.15$ is about 667 a second against the 2,000 still arriving. The queue in front grows until something sheds it.

Neither is a tuning problem.

Your average request is a fiction. Your users live in the tail.

The tail is the slow end of your response times: the few requests that take far longer than the middle.

A page that makes 100 parallel calls waits for the slowest one. Give each call a one-percent chance of being slow, and 63 percent of pages hit at least one. That is $1 - 0.99^{100}$.

More parallel calls turn a rare slow response into a common slow page.

SCALE KIT • WORKING THE TAIL

Measure the tail per dependency, and buy it back on a budget.

READ THE NUMBER RIGHT

Your p99 is the time 99 percent of requests come in under. A person makes dozens a day, so far more than one percent of people meet it.

HEDGING BUYS IT BACK

Waited longer than 95 percent normally take? Send a duplicate, take the first answer.

BUDGET THE HEDGE

Cap it at a few percent of traffic, or it floods you when you are already slow.

Each caching pattern owns a different failure.

Cache-aside

The app fills the cache on a miss. A cold key means every reader misses and rebuilds at once.

Read-through

The cache fetches for you. Cleaner code, and now the cache sits on the critical path.

Write-through

Write both together. Always fresh, and every write pays the cache's latency.

Write-behind

Write the cache, flush later. Fastest writes, and a crash loses them.

KEY INSIGHT

Choose by which failure you can survive, not by which pattern reads best in code.

Idempotency is what makes a retry safe, and retries are not optional.

An idempotent call runs twice and leaves the same result as running once. Networks duplicate, clients retry, queues redeliver. The only question is whether the second delivery is harmless or charges somebody twice.

THE RULE

Every mutating endpoint takes a client-supplied key and deduplicates on it.

THE KEY COMES FROM THE CALLER

Generated before the first attempt, reused on every retry of that same intent.

STORE THE RESULT, NOT THE FACT

A repeat returns the original answer, not an error saying it already happened.

All four must be true before you call any of this scalable.

01

The bottleneck is named and measured

Not suspected. A number, a graph, and the resource it belongs to.

02

Every queue is bounded

An unbounded queue turns a throughput problem into a memory outage.

03

Admission control sheds load on purpose

Refuse some at the door so the rest succeed — Maya's move at 15:50.

04

Adding a machine is routine

No manual steps, no rebalancing outage, no cold-cache stampede.

YOUR TURN

Your service takes 1,200 requests a second at 40 milliseconds each. Size it.

First: how many requests are in flight at once? Then a dependency slows and each request now spends 160 milliseconds inside, while the same 1,200 keep arriving.

Second: say what happens with an unbounded pool, and what happens with the pool your first answer sized. Do the arithmetic before you turn the page.

ONE ANSWER

One number, then two failures that look nothing alike.

01 In flight at once $1,200 \times 0.04$ is 48 — a floor on a good day, not your setting.

02 Unbounded $1,200 \times 0.16$ is 192 in flight. The machine you sized for 48 runs out of memory, and nothing warned you on the way.

03 Bounded at 48 Throughput falls to $48 / 0.16$, about 300 a second against 1,200 arriving. Nine hundred a second pile up.

Scale asks what happens when there is more.

Chapter nine asks what happens when one part of it stops. That is the whole difference between the two kits, and it is why admission control shows up in both of them.

